



Lista de Exercícios 8

8 de abril de 2022

Engenharia de Computação

Lilian Giselly Pereira Santos

Pedro Henrique de Brito Nascimento

Projeto e Análise de Algoritmos - Rian Gabriel Pinheiro

Capítulo 8 - Lidando com NP-Completeness

1. Um quadrado mágico de ordem 3 é uma tabela 3×3 preenchida com nove números inteiros distintos de 1 a 9, de modo que a soma dos números em cada linha, coluna e duas diagonais de ponta a ponta seja a mesma. Implemente um algoritmo backtracking e encontre todos os quadrados mágicos de ordem 3.

<u>Solução:</u>

```

int is_solution(int *matriz){
    int sum_l[3]={0}, sum_c[3]={0}, sum_pd=0, sum_sd=0;
    for (int i = 0; i < 9; i++)
    {
        int j = i%3;
        if(i==j) sum_pd += matriz[i];
        if(i+j==2) sum_sd += matriz[i];
        sum_l[i] += matriz[i];
        sum_c[j] += matriz[i];
    }
    if(sum_l[0]==sum_l[1] && sum_l[1]==sum_l[2] && sum_c[0]==sum_c[1]
    && sum_c[1]==sum_c[2] && sum_c[2]==sum_pd && sum_pd == sum_sd)
        return 1;
    return 0;
}

void generate(int matriz[9], int i, int element){
    if(i==9) return;
    if(element== 9 ) return;

    for (int j = 1; j <= 9; j++)
    {
        matriz[i]=j;
        if(is_solution(matriz)){
            print_mat(matriz);
        }
        generate(matriz, i+1, j);
    }
}

int main(){
    int matriz[9]={1};
    int element=1;
    generate(matriz,0,element);
}

```

2. Projete um algoritmo para o PROBLEMA DO CAMINHO HAMILTONIANO de um vértice fixo s.

Solução: O problema do caminho hamiltoniano pode ser resolvido através de *back-tracking*, que divide o problema total em subproblemas a fim de facilitar a sua resolução - pois solucionaremos pequenos problemas que resultarão na resolução de nosso principal objetivo.

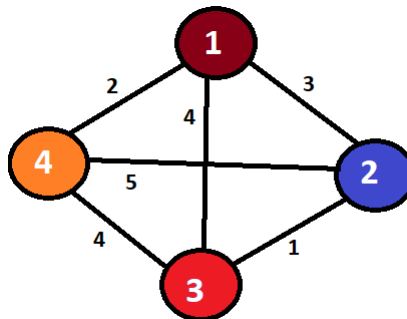
Com a divisão do problema do caminho hamiltoniano em subproblemas teremos uma nova instância dele, no qual teremos um subgrafo do original que começa em um dado vértice $v \in V$. Esse subgrafo nos permitirá construir progressivamente um caminho ao removermos vértices que já foram visitados.

Devemos definir os sub problemas nos pequenos grafos e, através deles, escolher os vértices que serão avaliados primeiro. Isso reduzirá a complexidade de espaço do algoritmo, visto que um único caminho ativo deve ser mantido na árvore dos possíveis

caminhos hamiltonianos. Para expandir o sub problema do caminho hamiltoniano começando em 's' em um grafo G, devemos quebrá-lo em subproblemas onde o i-ésimo subproblema será uma instância desse problema começando no i-ésimo vizinho de 's' em G.

3. Escolha um problema de otimização e dê um exemplo de uma execução de um algoritmo branch-and-bound sobre o problema escolhido. Escolha um instância pequena. Isso implica decidir:
- a) Como calcular um limitante para o problema?
 - b) Como você expandir um nó em subproblemas?
 - c) Qual subproblema escolher?

Solução: O problema escolhido será o TSP. O limitante pode ser calculado a partir da redução das linhas e colunas da matriz de adjacência e terá um custo de pelo menos o custo acumulado para reduzir tanto as linhas, quanto as colunas. O processo de redução é simplesmente calcular o valor mínimo de cada linha e subtrair de cada elemento, bem como calcular o valor mínimo de cada coluna e subtrair dos mesmos. O custo acumulado de cada linha e cada coluna é o limitante inferior para o custo do caminho do caixeiro viajante. Já o limitante superior é inicialmente infinito. O critério de escolha dos subproblemas será o custo acumulado do caminho, até o subproblema em questão da árvore gerada no problema do TSP. O subproblema a se escolher é, portanto, aquele que minimiza o custo da viagem do caixeiro. Considere a seguinte instância do TSP:

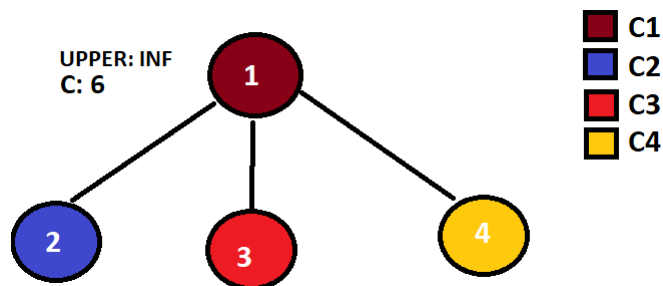


Para estimar o custo inicial do tour, temos que:

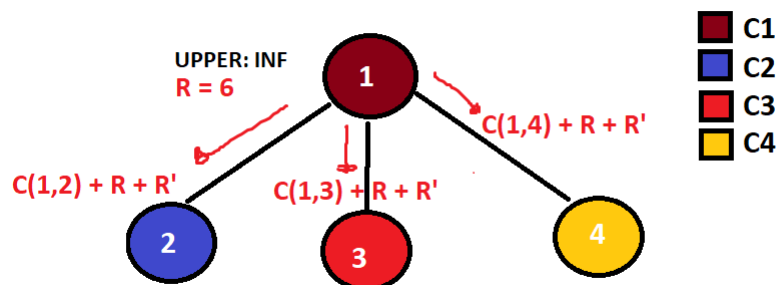
	1	2	3	4	MIN	
1	INF	3	4	2	2	
2	3	INF	1	5	1	
3	4	1	INF	4	1	
4	2	5	4	INF	2	
MIN	-	-	-	-	-	<-CUSTO

	1	2	3	4	MIN	
1	INF	1	2	0	2	
2	2	INF	0	4	1	
3	3	0	INF	3	1	
4	0	3	2	INF	2	
MIN	0	0	0	0	6+0 = 6	<-CUSTO

CUSTO DA REDUÇÃO = 6 (LIMITE INFERIOR)



Calcule agora o custo do próximo nó, que é o custo para ir do seu predecessor até ele mais o custo das reduções de linha e coluna anteriores, mais o custo da redução de linha e coluna do caminho atual. Para cada nó j sucessor de 1, faça a linha 1 do sucessor j ser toda infinita a partir da matriz reduzida do nó 1, bem como a coluna 2 também e assim sucessivamente para todos os nós vizinhos.



Para o nó 2 temos:

	1	2	3	4	MIN	
1	INF	INF	INF	INF	0	
2	2	INF	0	4	0	
3	0	INF	INF	0	3	
4	0	INF	2	INF	0	
MIN	0	0	0	0	3	<-CUSTO

$$C(C_1, C_2) + R + R' = 1 + 6 + 3 = 10$$

Para o nó 3 temos:

	1	2	3	4	MIN	
1	INF	INF	INF	INF	0	
2	0	INF	INF	0	0	
3	3	0	INF	3	0	
4	0	3	INF	INF	0	
MIN	0	0	0	2	2	<-CUSTO

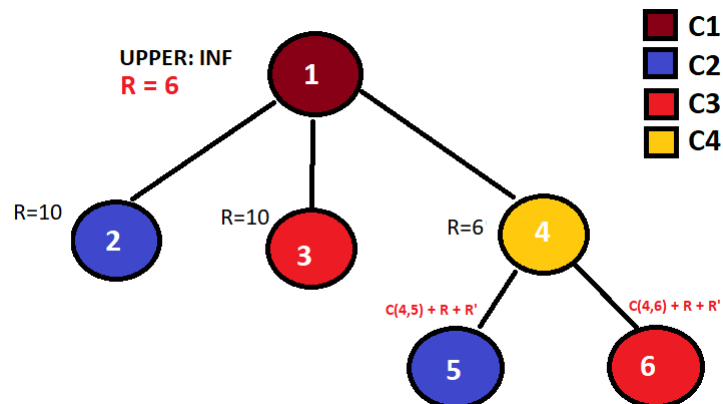
$$C(C_1, C_3) + R + R' = 2 + 6 + 2 = 10$$

Para o nó 4 temos:

	1	2	3	4	MIN	
1	INF	INF	INF	INF	0	
2	2	INF	0	INF	0	
3	3	0	INF	INF	0	
4	0	3	2	INF	0	
MIN	0	0	0	0	0	<-CUSTO

$$C(C_1, C_4) + R + R' = 0 + 6 + 0 = 6$$

Como a estratégia adotada é do Least Cost Branch and Bound, o nó a ser escolhido será o nó 4. Repetindo as estratégias citadas anteriormente e gerando os sucessores. Além disso, os passos anteriores de redução e cálculo de custo se repetem, agora, a partir do nó 4.



Para o nó 5, temos que:

	1	2	3	4	MIN	
1	INF	INF	INF	INF	0	
2	INF	INF	0	INF	0	
3	0	INF	INF	INF	3	
4	INF	INF	INF	INF	0	
MIN	0	0	0	0	3	<-CUSTO

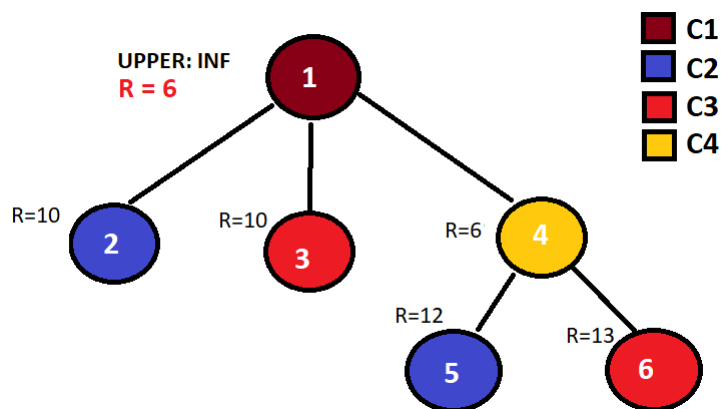
$$C(C_4, C_2) + R + R' = 3 + 6 + 3 = 12$$

Para o nó 6, temos que:

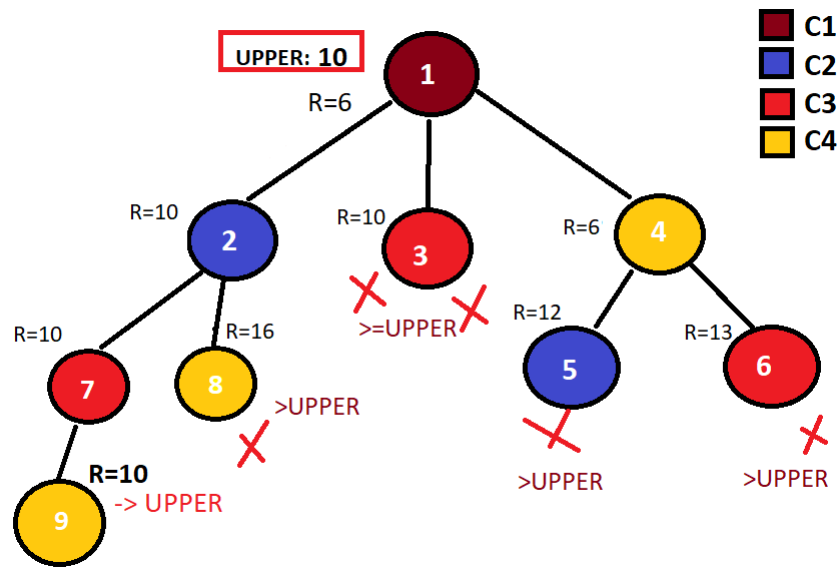
	1	2	3	4	MIN	
1	INF	INF	INF	INF	0	
2	0	INF	INF	INF	2	
3	0	0	INF	INF	3	
4	INF	INF	INF	INF	0	
MIN	0	0	0	0	5	<-CUSTO

$$C(C_4, C_3) + R + R' = 2 + 6 + 5 = 13$$

Temos a seguinte situação:



A título de simplificação, ocultaremos os próximos passos que são análogos e obtemos o seguinte resultado:.



É possível verificar que ao obter uma solução candidata todas as outras ramificações que existiam anteriormente possuíam custos maiores ou iguais à solução encontrada, logo elas foram descartadas por meio da poda.

4. De forma similar à questão anterior (responda as perguntas!), projete um algoritmo branch-and-bound para o problema da COBERTURA DE CONJUNTO.

Solução: Considere o grafo $G=(V,E)$. O algoritmo pode ser pensado como uma busca em profundidade em uma árvore, na qual o mesmo busca pela solução ótima recursivamente criando os galhos para um vértice e a partir do mesmo criando dois vértices até que não haja mais vértices em G . Ou seja, quando o algoritmo ramifica um vértice, ele divide o espaço das soluções em um conjunto de subconjuntos menores calculando assim os limitantes superiores e inferiores relativos para cada nó, reduzindo assim o espaço de busca.

A entrada para o algoritmo é um grafo G , um limitante superior $UpperB$ e uma solução candidata S que representa uma cobertura de vértices parcial em construção. O limitante superior é uma superestimação do tamanho da cobertura mínima de vértices do subgrafo e pode ser obtido calculando o tamanho da cobertura mínima de vértices encontrada até o momento.

Quando o tamanho de S (solução candidata atual da cobertura de vértices) somado ao limitante inferior do subgrafo se tornar maior ou igual ao limitante superior, a poda é então realizada na árvore de busca e o limitante superior $UpperB$ que é o tamanho da cobertura mínima de vértices é retornado.

Para calcular o limitante inferior 3 métodos podem ser utilizados clique_bound, degree_bound, sat_bound.

- clique_bound: Faz uma decomposição do grafo em um conjunto de cliques disjuntas $C_1, C_2, \dots, C_r$ e soma os tamanhos das cliques, de modo que o limitante inferior é:

$$clique_{bound} = \sum_{i=1}^{i=r} (|C_i| - 1)$$

- degree_bound: Seleciona o vértice com maior grau, nomeando-o v'_1 e atualiza o grau dos outros vértices, seleciona agora o vértice de maior grau e o nomeia v'_2 e assim sucessivamente. Busca-se então um vértice v'_i tal que

$$d(v'_1) + d(v'_2) + \dots + d(v'_i) \geq |E|$$

$$d(v'_1) + d(v'_2) + \dots + d(v'_{i-1}) < |E|$$

Assim o limitante inferior pode ser dado por:

$$degree_{bound} = \lfloor i + \frac{|E|}{d(v'_i + 1)} \rfloor, \quad v'_{i+1} \in H$$

onde $G \setminus \{v'_1, v'_2, \dots, v'_i\} = (V', E')$

- sat_bound: Reduz o grafo $G = (V, E)$ a uma instância MaxSAT e se G pode ser particionado em k cliques disjuntas e existem s subconjuntos inconsistentes na instância MaxSAT, então o limitante inferior é dado por:

$$sat_{bound} = |V| + k + s$$

Assim, o algoritmo pode ser dado por:

```
def minimum_vertex_cover_b_bound(G, UpperB, S):
    if len(S) + max(degree_bound(G), clique_bound(G), sat_bound(G)) >= UpperB:
        return UpperB
    if len(G) == 0:
        return len(S)
    v = select_max_dg_vert(G)
    sol1 = minimum_vertex_cover_b_bound(remove(G, Neighbourhood(v)), UpperB, S.append(Neighbourhood(v)))
    sol2 = minimum_vertex_cover_b_bound(remove(G, v), UpperB, S.append(v))

    return min(sol1, sol2)
```

5. No problema da ÁRVORE DE STEINER MÍNIMA, a entrada consiste em: um grafo completo $G = (V, E)$ com distâncias d_{uv} entre todos os pares de nós; e um determinado conjunto de vértices terminais $V \subseteq V'$. O objetivo é encontrar uma árvore de custo mínimo que inclua os vértices V . Essa árvore pode ou não incluir nós em $V \setminus V'$. Sabe-se que este problema é NP-difícil, proponha uma heurística construtiva para gerar uma solução viável para o problema (note que a solução pode não ser ótima).

Solução: Iremos supor que a distância entre as entradas atende às definições de métrica e , com isso, provaremos que uma solução para a árvore de steiner mínima pode ser obtida através de um algoritmo de aproximação ao ignorarmos os nós não terminais e retornarmos a árvore geradora mínima em V' .

Para isso, consideremos T como a Árvore Steiner Mínima possuindo custo C e, com isso, podemos moldar a árvore steiner para obter custo $2C$ ao passar por todos os vértices. Sendo i, j e j, k vértices adjacentes no caminho, usamos a desigualdade triangular $d_{ik} \leq d_{ij} + d_{jk}$.

Com isso, nós podemos desviar de um vértice intermediário ' j ' e conectar ' i ' e ' k ' diretamente, sem aumentar o custo. Com essa técnica, podemos desviar de todos os vértices que não estão em V' e estão presentes no caminho, e de todos os vértices de V' que já foram visitados duas vezes. Isso resulta em um caminho de custo de no máximo $2C$, passando por todos os vértices de V' exatamente uma vez.

Portanto, esse caminho é uma árvore geradora para V' de custo $2C$, o que implica que o custo da Árvore de Steiner Mínima será menor ou igual a $2C$, assim nos dando a prova da aproximação desejada.

6. Implemente um algoritmo backtracking para o PROBLEMA DO PASSEIO DO CAVALO, iniciando de uma das quinas. Informe o tempo em segundos e apresente a solução.

Solução:

```

def print_solution_kt(board):
    for i in range(8):
        for j in range(8):
            print(board[i][j], end=' ')
        print()

def is_valid_move(board, x, y):
    # verifica se a posição eh valida de acordo com as limitações
    if(x >= 0 and y >= 0 and x < 8 and y < 8 and board[x][y] == -1):
        return True
    return False

def solve_kt_bt(board, curr_x, curr_y, next_x, next_y, pos):
    if pos == 64:
        return True

    # tenta todas as coordenadas através de backtracking
    for i in range(8):
        x = curr_x + next_x[i]
        y = curr_y + next_y[i]
        if(is_valid_move(board, x, y)):
            board[x][y] = pos
            if(solve_kt_bt(board, x, y, next_x, next_y, pos+1)):
                return True
            board[x][y] = -1
    return False

def knights_tour_bt():
    board = [[-1 for i in range(8)] for i in range(8)]
    board[0][0] = 0 # inicializa o cavalo na quina do primeiro bloco

    # prox coordenadas
    next_x = [2, 1, -1, -2, -2, -1, 1, 2]
    next_y = [1, 2, 2, 1, -1, -2, -2, -1]

    pos = 1

    # chama a função que vai executar o backtracking
    if solve_kt_bt(board, 0, 0, next_x, next_y, pos):
        print_solution_kt(board)
    else:
        print("There's no solution for this problem.")

knights_tour_bt()

```

Considerando a coordenada do primeiro movimento, a solução se dá a seguir:

```

0 37 58 35 42 47 56 51
59 34 1 48 57 50 43 46
38 31 36 41 2 45 52 55
33 60 39 26 49 54 3 44
30 9 32 61 40 25 22 53
17 62 27 10 23 20 13 4
8 29 18 15 6 11 24 21
63 16 7 28 19 14 5 12

```

O tempo em segundos decorrido para encontrar a solução foi: 41.19298219680786s. Importante observar que o tempo decorrido varia de acordo com o caminho que se percorre, caso invertêssemos a ordem entre *next_x* e *next_y* teríamos encontrado uma

solução em 37.05153942108154s. Além disso, a complexidade para esse algoritmo é $O(8^{N^2})$.

7. Apresente o algoritmo de busca local 2-opt para o TSP, dê um exemplo de execução e mostre que ele não é exato.

Solução: Supondo que o DFS escolha arbitrariamente um vértice inicial para construir o caminho em pré-ordem e simultaneamente em ordem. Temos o seguinte algoritmo:

```
def two_opt_tsp(complete_graph):  
    mst_pre_solution = prim(complete_graph)  
  
    constructed_solution = dfs(mst_pre_solution)  
    solution = delete_duplicate_from_tour(constructed_solution)  
  
    return solution
```

O algoritmo é 2-OPT pois o custo para achar um caminho que visita todos os vértices exatamente uma vez, no pior dos casos, é o custo de executar o DFS em cada vértice da MST. Neste sentido, cada aresta da MST é verificada 2 vezes. Assim, o custo é dado por:

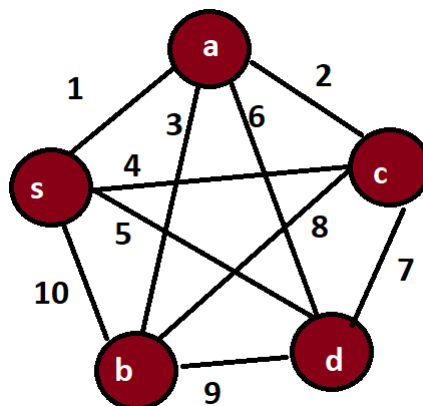
$$OPT \geq MST$$

$$MST_{trilha} \leq 2MST$$

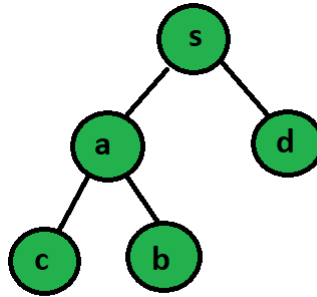
$$\therefore MST_{Trilha} \leq 2 \cdot OPT$$

Ele não é exato pois depende da escolha e construção da MST e da eliminação dos vértices duplicados. Nem sempre escolhe o ótimo global, dado que a troca de vértices constrói um ótimo local favorável à MST.

Exemplo:



$MST : (s \rightarrow a), (a \rightarrow c), (s \rightarrow d), (a \rightarrow b)$



DFS (Pré/Em Ordem): $s \rightarrow a \rightarrow c \rightarrow a \rightarrow b \rightarrow s \rightarrow d \rightarrow s$

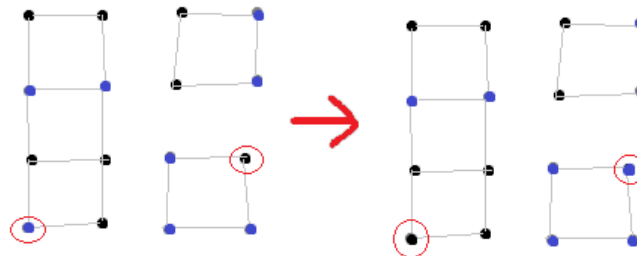
Removendo duplicatas: $s \rightarrow a \rightarrow c \rightarrow \underline{a} \rightarrow b \rightarrow \underline{s} \rightarrow d \rightarrow s$

Solução Caixeiro[2-OPT]: $s \rightarrow a \rightarrow c \rightarrow b \rightarrow d \rightarrow s$ CUSTO: 25

Solução Caixeiro[Ótimo]: $s \rightarrow c \rightarrow a \rightarrow b \rightarrow d \rightarrow s$ CUSTO: 23

8. Escolha um problema de otimização NP-difícil e apresente um estrutura de vizinhança para tal problema

Solução: O problema escolhido é conhecido por "Partição de Grafos" e consiste em retornar uma partição dos vértices em dois grupos A e B de acordo com a entrada que consiste em um grafo e um número. Esse é um problema NP-difícil e para estruturar a sua vizinhança, podemos seguir com a opção (A, B), sendo $|A| = |B|$. Os vizinhos serão todas as soluções obtidas ao trocarmos um par de vértices através dos cortes, ou seja, a vizinhança é composta por soluções da forma $(A - a + b, B - b + a)$, onde $a \in A$ e $b \in B$. Um exemplo desse movimento é:



No entanto, é possível observar que essa solução não é ótima e uma melhoria que pode ser feita é a expansão do tamanho da sua vizinhança a fim de permitir duas trocas por vez.

9. Escolha 5 meta-heurísticas do livro "Handbook of Metaheuristics"

([https:// www.springer.com/gp/book/9783319910857](https://www.springer.com/gp/book/9783319910857), cada capítulo fala de uma meta-heurística). Para cada uma das 5, descreva o algoritmo e explique quais os mecanismos de intensificação e diversificação da busca.

Solução:

- **Têmpera Simulada (Simulated Annealing):**
Importante ressaltar a analogia do comportamento do algoritmo ao seu nome, que representa uma forma de "abaixar a temperatura" gradualmente até que um líquido se solidifique. No nosso caso, a temperatura serve como métrica para intensificação e diversificação, e representa uma probabilidade de escolher aleatoriamente que varia com o tempo, em outras palavras, é uma forma de deixar a escolha da probabilidade de forma dinâmica e gradual. É no contexto inicial desse algoritmo que ocorre a maior quantidade de diversificação, isto é, exploração a fim de conhecer as possibilidades e evitar cenários como máximos locais, planícies e encostas. Em seguida, ocorre a intensificação, passo que busca otimizar certo resultado.
- **Busca Tabu (Tabu Search):**
É um algoritmo de busca por vizinhança que implementa o conceito de memória para o seu funcionamento. Com a memória, ele permite que haja maior diversificação das soluções no espaço de busca ao evitar que haja repetição à curto prazo de movimentos realizados recentemente e obrigando a procura de novas soluções ao sair do espaço de vizinhança. Esse algoritmo foi criado pois apesar do fator de aleatoriedade ser crucial para fugir de cenários como máximos locais, planícies e encostas, isso pode fazer com que haja uma taxa significativa de visita em soluções repetitivas. Ele funciona partindo de uma solução inicial e se movendo, a cada iteração, para a melhor solução na vizinhança sem ir para soluções que estão na lista tabu, dessa forma, intensificando e diversificando as soluções.
- **Variable Neighborhood Search:**
É um algoritmo que possui mais de uma vizinhança auxiliando na fuga de um máximo local. Inicialmente, uma solução é gerada automaticamente. Após isso, ela é intensificada através do VND e verificada se é a melhor solução que já foi encontrada. Caso seja, ela explora a vizinhança em questão, caso não seja, ela explora outra (realizando uma visita aleatória também). Isso garante que haja diversificação e intensificação da busca. Uma característica importante é que deve haver diferença significativa entre as vizinhanças.
- **Algoritmo Genético (Genetic Algorithms):**
É um algoritmo de otimização baseado em evolução natural. Para que a solução "ótima" seja encontrada, é necessário que se tenha várias "gerações", assim como ocorre na vida real. Há dois processos importantíssimos nesse algoritmo,

sendo eles: cross-over e mutação. O primeiro é responsável por misturar as informações de maneira aleatória a fim de criar novas soluções que são a mistura de soluções já existentes, essa seria a etapa de diversificação. Já a mutação é responsável por adicionar variedade às soluções existentes, adicionando pequenas mudanças, essa sendo a intensificação. A mutação também é responsável por casos onde são criadas características que caracterizam um melhor usuário.

- Colônia de formigas (Ant Colony Optimization):

Esse algoritmo inspirado na comunicação baseada em feromônios realizada pelas formigas é geralmente utilizado para encontrar o melhor caminho, visto que a analogia procede da grande habilidade do inseto em se guiar em busca de comida através da cooperação entre a colônia. O algoritmo funciona baseado na comunicação dentro de uma colônia de formigas artificiais, usando o caminho artificial de feromônios deixados por elas. Esse caminho é usado de maneira numérica para probabilisticamente construir soluções e, em seguida, adaptar as soluções existentes de acordo com a experiência adquirida. Inicialmente sem muita experiência, as formigas procuram diversificar seus caminhos, explorando as possibilidades e à medida que a experiência de formigas anteriores é adquirida, a colônia recebe conhecimento e consegue se guiar para, baseada nele, intensificar e encontrar a melhor solução possível.

10. No problema do BIN PACKING, a entrada consiste em: n itens com tamanhos $s_1, s_2, \dots, s_n$ em que $s_i \in [0, 1]$. O objetivo é encontrar o menor número de “bins” (caixas) unitárias para armazenar os n itens. O algoritmo First Fit consiste em:

```

1: procedure ALGORITMO FIRST FIT(vetor  $s[1, \dots, n]$ )
2:   for  $i \leftarrow 1 \rightarrow n$  do
3:     Coloque o item  $i$  no bin de menor índice que tenha espaço disponível  $\geq s_i$ 
4:   end for
5: end procedure

```

Mostre que o algoritmo é 2-aproximado. Dica: O FF não deixa, ao final, dois bins com espaço utilizado $\leq 0,5$

Solução:

Suponha que o algoritmo First-Fit utilize m caixas com $m = \{W_1, W_2, \dots, W_m\}$. Então, ao menos $(m - 1)$ caixas estão mais do que meio cheias. Isto é fácil ver, pois se há 2 caixas cuja capacidade $W_i < \frac{W_{imax}}{2}$ e $W_j < \frac{W_{jmax}}{2}$, não supera metade

da capacidade máxima das caixas correspondentes, então, pelo algoritmo First-Fit, como $W_{imax} = W_{jmax}$, os itens poderiam ser tirados da segunda caixa e colocados na primeira utilizando o First-Fit pois haveria capacidade, restando apenas 1 caixa ("m - 1") cuja capacidade alocada é, agora, maior que a metade da capacidade máxima.

Sendo assim, supondo que o algoritmo First-Fit utiliza m caixas, então no mínimo $m - 1$ caixas utilizam mais do que metade da capacidade máxima. Portanto:

$$OPT \geq \sum_{i=1}^{i=n} s_i > \frac{m-1}{2}$$

$$OPT \geq \frac{m-1}{2}$$

$$2 \cdot OPT \geq m - 1$$

$$\therefore 2 \cdot OPT \geq m$$

Assim, o bin-packing com estratégia First-Fit é dito 2-aproximado, pois no pior dos casos, o custo para construir a solução é 2 vezes o custo da solução ótima.